

TruthLens — Pseudocode

Fake News Detection | Python (Flask, scikit-learn, NLTK) · React (Vite, TypeScript)

1. TextPreprocessor

```
CLASS TextPreprocessor:
  INIT:
    stop_words ← NLTK stopwords MINUS {"not", "no", "nor", "never"}
    lemmatizer ← WordNetLemmatizer()

  FUNCTION preprocess(text) → String:
    text ← LOWERCASE(text)
    text ← REMOVE URLs, HTML tags, emails, special characters
    tokens ← word_tokenize(text)
    tokens ← FILTER tokens NOT IN stop_words AND LENGTH > 1
    tokens ← [ lemmatizer.lemmatize(t) FOR t IN tokens ]
    RETURN JOIN(tokens, " ")

  FUNCTION get_preprocessing_steps(text) → Dict:
    RETURN { original, after_cleaning, tokens,
             after_stopword_removal, after_lemmatization,
             final_text, word_counts }
```

2. NaiveBayesClassifier

```
CLASS NaiveBayesClassifier:
  INIT:
    vectorizer ← TfidfVectorizer(max_features=5000, ngram_range=(1,2))
    model      ← MultinomialNB()

  FUNCTION fit(texts, labels):
    X ← vectorizer.fit_transform(texts)
    model.fit(X, labels)

  FUNCTION predict_proba(texts) → [[P(REAL), P(FAKE)]]:
    X ← vectorizer.transform(texts)
    RETURN model.predict_proba(X)

  FUNCTION save / load(paths):
    joblib.dump / joblib.load → naive_bayes.pkl, tfidf.pkl
```

3. Training Pipeline (train.py)

```

BEGIN Train:
  df ← READ_CSV("data/dataset.csv")
  df["text"] ← df["title"] + " " + df["text"]
  df ← SHUFFLE → DROP nulls

  preprocessor ← TextPreprocessor()
  processed ← [ preprocessor.preprocess(t) FOR t IN df["text"] ]

  vectorizer ← TfidfVectorizer(max_features=2000, ngram=(1,1),
                                max_df=0.7, min_df=2)
  X ← vectorizer.fit_transform(processed)

  X_train, X_test, y_train, y_test ← train_test_split(X, labels,
                                                         test_size=0.20, stratify=labels)

  model ← MultinomialNB()
  model.fit(X_train, y_train)

  // Evaluate
  predictions ← model.predict(X_test)
  COMPUTE accuracy, precision, recall, F1, confusion_matrix

  // Save
  joblib.dump → naive_bayes.pkl, tfidf.pkl
  WRITE_JSON → metrics.json
END

```

4. Flask REST API (app.py)

```

INIT: app ← Flask() | CORS(app) | load model from .pkl

GET /api/health          → { status, model_loaded, model_type }
GET /api/metrics          → READ metrics.json
GET /api/training-history → { epochs, train_accuracy, val_accuracy,
                             train_loss, val_loss }

POST /api/predict:
  INPUT: { text }
  IF LENGTH(text) < 20 → RETURN 400

  IF model is NULL → RETURN demo_predict(text)  // keyword heuristic

  processed ← preprocessor.preprocess(text)
  [P(REAL), P(FAKE)] ← model.predict_proba([processed])

  IF P(FAKE) >= 0.80 → verdict ← "FAKE",      conf ← P(FAKE)
  ELIF P(REAL) >= 0.80 → verdict ← "REAL",    conf ← P(REAL)
  ELSE                → verdict ← "SUSPICIOUS", conf ← MAX(P(FAKE), P(REAL))

```

```
RETURN { prediction, confidence, probability_fake,
        probability_real, preprocessing_steps }
```

5. React Frontend (Index.tsx)

```
COMPONENT Index:
  STATE: isProcessing=FALSE, result=NULL, activeTab="detect"

  FUNCTION handleAnalyze(text):
    SET isProcessing ← TRUE
    prediction ← AWAIT POST /api/predict { text }
    SET result ← prediction
    addToHistory(text, prediction.verdict, prediction.confidence)
    SHOW TOAST ("✅ Verified" OR "🚨 Fake Detected")
    SET isProcessing ← FALSE

  RENDER Tabs:
    "detect" → <NewsInput> + <PredictionResultCard result>
    "metrics" → <ModelMetrics> (fetches GET /api/metrics)
    "how" → <HowItWorks> (static pipeline visualization)
    "history" → <HistoryPanel> (in-memory history list)
```

Decision Engine Summary

Condition	Verdict	Confidence
$P(\text{FAKE}) \geq 0.80$	FAKE	$P(\text{FAKE})$
$P(\text{REAL}) \geq 0.80$	REAL	$P(\text{REAL})$
Neither	SUSPICIOUS	$\text{MAX}(P(\text{FAKE}), P(\text{REAL}))$